

system-testing-post-fixes

June 3, 2026

1 System Testing Update

1.1 Introduction

During the previous round of system testing, an error was identified that produced a mismatch between the manually derived results and the results generated by the implemented system. The underlying defect has since been corrected, and the associated inconsistencies have been resolved. Accordingly, the test procedure is repeated in this document to verify that the system now operates as intended and produces results consistent with the expected calculations.

1.2 The Problem

The discrepancy originated from the system's aggregation logic for participant weights. Specifically, the implementation incorrectly treated each submission as an independent voting entity rather than associating submissions with their corresponding stakeholder group contribution. In the original workflow, each submission was transformed into an AHP result, after which its pairwise comparison matrix and normalized voting weight were appended directly to the aggregation set. The system then renormalized participant weights across this expanded set and applied a single weighted geometric mean over all individual matrices. This approach introduced unintended distortion in the aggregate result by effectively overrepresenting participants with multiple submissions.

1.3 Testing

To evaluate the corrected implementation, the same test cases are executed again under identical conditions. Reapplying the original test set enables direct comparison between the revised system output and the previously established hand-calculated results, thereby confirming whether the correction eliminates the prior inconsistency and preserves the expected outcome.

1.3.1 Test 1

AHP and Pairwise Matrix

Group AHP Weights

	Criterion	Group AHP Weight
0	implementation_cost	0.2315
1	expected_crime_reduction	0.2598
2	community_trust_impact	0.2775
3	implementation_time	0.2312

Aggregated Group Pairwise Matrix

	implementation_cost	expected_crime_reduction	community_trust_impact	implementation_time
implementation_cost	1	0.891	0.8341	1.0012
expected_crime_reduction	1.1223	1	0.936	1.1236
community_trust_impact	1.199	1.0683	1	1.2004
implementation_time	0.9988	0.89	0.8331	1

Group Consistency Ratio

0.0

Group Matrix Consistent?

Yes

System

Manual Derived

=== Final Aggregated Pairwise Matrix ===

```
[[1.          0.89104228 0.83405895 1.00118653]
 [1.1222812  1.          0.93604868 1.12361282]
 [1.19895602 1.0683205  1.          1.20037862]
 [0.99881488 0.88998628 0.83307049 1.          ]]
```

=== AHP Weights ===

cost: 0.2315

crime_reduction: 0.2598

community_trust: 0.2775

months_to_implement: 0.2312

Consistency Ratio (RC): 0.0000

TOPSIS

Group TOPSIS / Fuzzy TOPSIS Ranking

Method	Alternatives	Top Alternative	Score Spread
TOPSIS	4	more_patrols	0.152

Alternative	TOPSIS Score	Rank
more_patrols	0.5752	1
community_prevention	0.5750	2
expand_precinct	0.4834	3
new_precinct	0.4232	4

System

Topsis Rankings:		
	 alternative	# Topsis Score
2	more_patrols	0.575172904455181
3	community_prevention	0.5750274311156185
1	expand_precinct	0.48343096520073364
0	new_precinct	0.4231852123299354

Manual Derived

1.3.2 Test 2

AHP and Pairwise Matrix

Group AHP Weights

	Criterion	Group AHP Weight
0	implementation_cost	0.2373
1	expected_crime_reduction	0.2446
2	community_trust_impact	0.279
3	implementation_time	0.2391

Aggregated Group Pairwise Matrix

	implementation_cost	expected_crime_reduction	community_trust_impact	implementation_time
implementation_cost	1	0.9703	0.8507	0.9925
expected_crime_reduction	1.0306	1	0.8767	1.0229
community_trust_impact	1.1755	1.1406	1	1.1667
implementation_time	1.0075	0.9776	0.8571	1

Group Consistency Ratio

0.0

Group Matrix Consistent?

Yes

System

Manual Derived

```
=== Final Aggregated Pairwise Matrix ===
[[1.          0.97031585 0.85069966 0.99252079]
 [1.03059226 1.          0.87672449 1.02288424]
 [1.17550299 1.14060918 1.          1.16671116]
 [1.00753557 0.97762773 0.85711017 1.          ]]
```

```

=== AHP Weights ===
cost: 0.2373
crime_reduction: 0.2446
community_trust: 0.2790
months_to_implement: 0.2391

```

Consistency Ratio (RC): -0.0000

TOPSIS

Method: TOPSIS Alternatives: 4 Top Alternative: more_patrols Score Spread: 0.1935

Alternative	TOPSIS Score	Rank
more_patrols	0.5960	1
community_prevention	0.5956	2
expand_precinct	0.4963	3
new_precinct	0.4025	4

System

Topsis Rankings:		
	alternative	# Topsis Score
2	more_patrols	0.5959911659278312
3	community_prevention	0.5955696020971294
1	expand_precinct	0.49633455563837614
0	new_precinct	0.40245708824604126

Manual Derived

1.3.3 Test 3

AHP and Pairwise Matrix

Group AHP Weights

	Criterion	Group AHP Weight
0	implementation_cost	0.2441
1	expected_crime_reduction	0.2458
2	community_trust_impact	0.2726
3	implementation_time	0.2375

Aggregated Group Pairwise Matrix

	implementation_cost	expected_crime_reduction	community_trust_impact	implementation_time
implementation_cost	1	0.9935	0.8957	1.0279
expected_crime_reduction	1.0066	1	0.9016	1.0346
community_trust_impact	1.1164	1.1091	1	1.1475
implementation_time	0.9729	0.9665	0.8714	1

Group Consistency Ratio

0.0

Group Matrix Consistent?

Yes

System

Manual Derived

=== Final Aggregated Pairwise Matrix ===

```
[[1.          0.99346723 0.89570856 1.02786741]
 [1.00657573 1.          0.9015985  1.03462639]
 [1.11643457 1.10914115 1.          1.14754671]
 [0.97288813 0.96653247 0.87142423 1.          ]]
```

=== AHP Weights ===

cost: 0.2441

crime_reduction: 0.2458

community_trust: 0.2726

months_to_implement: 0.2375

Consistency Ratio (RC): 0.0000

TOPSIS

Method

TOPSIS

Alternatives

4

Top Alternative

more_patrols

Score Spread

0.1996

Alternative	TOPSIS Score	Rank
more_patrols	0.5990	1
community_prevention	0.5989	2
expand_precinct	0.5009	3
new_precinct	0.3994	4

System

Topsis Rankings:		
	alternative	# Topsis Score
2	more_patrols	0.5989671223125447
3	community_prevention	0.598854768751995
1	expand_precinct	0.500915022204023
0	new_precinct	0.39940469205356627

Manual Derived

1.4 Results

The results of all three test cases demonstrate parity between the manual calculations and the system-generated outputs. In each case, the pairwise comparison matrices, AHP weights, and TOPSIS rankings are identical across both evaluation methods.

Test Case 1 evaluated a baseline scenario with one valid participant from each stakeholder group, for a total of five participants. Test Case 2 involved 18 participants and examined disagreement across multiple stakeholder groups under conditions of business influence without full dominance. Test Case 3 involved 29 participants, of which 34.5% belonged to the business/economic stakeholder group, and was designed to assess a business-skewed participation imbalance in which local business owners heavily influenced prioritization toward rapid crime-reduction objectives.

Because the results are consistent across all test cases, the corrected system can be considered to be functioning as intended for the validated scenarios. With this result established, the next step is sensitivity testing to evaluate how variations in stakeholder preferences and participation patterns affect the final outcomes.

1.5 Code

```
[13]: import pandas as pd
import numpy as np

from pyDecision.algorithm import topsis_method, ahp_method
from IPython.display import display

def load_data(file_path):
    try:
        return pd.read_csv(file_path)
    except Exception as e:
        print(f"Error loading data: {e}")
        return None
```

```

def _convert_preference(pref):
    if pref == "Very Low":
        return 1
    elif pref == "Low":
        return 2
    elif pref == "Medium":
        return 3
    elif pref == "High":
        return 4
    elif pref == "Very High":
        return 5
    else:
        return None

def convert_linguistic_to_numeric(df):
    df["implementation_cost"] = df["implementation_cost"].
    ↪apply(_convert_preference)
    df["expected_crime_reduction"] = df["expected_crime_reduction"].
    ↪apply(_convert_preference)
    df["community_trust_impact"] = df["community_trust_impact"].
    ↪apply(_convert_preference)
    df["implementation_time"] = df["implementation_time"].
    ↪apply(_convert_preference)
    return df

def create_individual_pairwise_matrix(df):
    """
    Creates pairwise comparison matrices grouped by stakeholder_group.

    For each stakeholder group:
    - Builds individual 4x4 pairwise matrices for each stakeholder
    - Aggregates multiple stakeholders in the same group using geometric mean

    Returns a dict mapping stakeholder_group to aggregated 4x4 pairwise matrix.

    Args:
        df: DataFrame with columns [stakeholder_group, implementation_cost,
    ↪expected_crime_reduction,
                                   community_trust_impact, implementation_time]

    Returns:
        dict: {stakeholder_group: aggregated_pairwise_matrix_4x4}
    """
    criteria_cols = ["implementation_cost", "expected_crime_reduction",
    ↪"community_trust_impact", "implementation_time"]

```

```

# Group by stakeholder_group
grouped = df.groupby("stakeholder_group")

result = {}

for stakeholder_group, group_df in grouped:
    # Extract numeric values for this stakeholder group
    group_data = group_df[criteria_cols].to_numpy() # shape:  $\hookrightarrow$ 
     $\hookrightarrow$  (n_individuals, 4)

    # Build individual pairwise matrices for each stakeholder in the group
    individual_matrices = []

    for row in group_data:
        # Create 4x4 pairwise matrix: matrix[i][j] = row[i] / row[j]
        pairwise_matrix = np.zeros((4, 4))
        for i in range(4):
            for j in range(4):
                pairwise_matrix[i][j] = float(row[i] / row[j])
            individual_matrices.append(pairwise_matrix)

    # Aggregate using geometric mean
    if len(individual_matrices) == 1:
        # Single stakeholder in group: use as-is
        aggregated_matrix = individual_matrices[0]
    else:
        # Multiple stakeholders: compute geometric mean element-wise
        # Geometric mean: (product)  $^{\wedge}$  (1/n)
        aggregated_matrix = np.ones((4, 4))
        for i in range(4):
            for j in range(4):
                product = 1.0
                for individual_matrix in individual_matrices:
                    product *= individual_matrix[i][j]
                aggregated_matrix[i][j] = product ** (1.0 /  $\hookrightarrow$ 
 $\hookrightarrow$  len(individual_matrices))

    result[stakeholder_group] = aggregated_matrix

return result

def create_aggregate_pairwise_matrix(grouped_matrices, stakeholder_weights):
    """
    Aggregates pairwise matrices across stakeholder groups using weighted  $\hookrightarrow$ 
     $\hookrightarrow$  geometric mean.

```



```

Args:
    grouped_matrices: dict mapping stakeholder_group to 4x4 pairwise matrix
    stakeholder_weights: dict mapping stakeholder_group to voting_power
    ↪ (0-1)

Returns:
    numpy array: 4x4 aggregated pairwise matrix
    """
    # Validate that weights sum to 1 (approximately)
    weights_sum = sum(stakeholder_weights.values())
    if weights_sum == 0:
        raise ValueError("Stakeholder weights sum to 0")

    # Renormalize weights to sum to 1
    normalized_weights = {k: v / weights_sum for k, v in stakeholder_weights.
    ↪ items()}

    # Initialize aggregated matrix with ones
    aggregated_matrix = np.ones((4, 4))

    # For each matrix position, compute weighted geometric mean
    for i in range(4):
        for j in range(4):
            product = 1.0
            for stakeholder_group, matrix in grouped_matrices.items():
                weight = normalized_weights.get(stakeholder_group, 0)
                if weight > 0:
                    # Use weight as exponent in geometric mean formula
                    product *= matrix[i][j] ** weight
            aggregated_matrix[i][j] = product

    return aggregated_matrix

def test1():
    """Test 1: Basic functionality with 5 stakeholders, one per group."""
    test1_raw = {
        "participant": ["Jordan Reed", "Maya Chen", "Sofia Alvarez", "Ethan",
        ↪ "Brooks", "Ari Morgan"],
        "stakeholder_group": ["community_residents", "business_economic",
        ↪ "police_leadership", "city_government", "community_advocates"],
        "implementation_cost": ["High", "Medium", "Low", "High", "Medium"],
        "expected_crime_reduction": ["Medium", "Very High", "Very High",
        ↪ "High", "Low"],
        "community_trust_impact": ["Very High", "Low", "Medium", "High", "Very",
        ↪ "High"],
    }

```

```

        "implementation_time": ["Medium", "Very High", "High", "Medium", "Low"]
    }

    df = pd.DataFrame(test1_raw)
    df = convert_linguistic_to_numeric(df)

    # Create individual pairwise matrices grouped by stakeholder_group
    grouped_matrices = create_individual_pairwise_matrix(df)

    print("\n=== Grouped Pairwise Matrices by Stakeholder Group ===")
    for stakeholder_group, matrix in grouped_matrices.items():
        print(f"\n{n{stakeholder_group}}:")
        print(matrix)

    # Define stakeholder voting power (from scenario.json)
    stakeholder_weights = {
        "community_residents": 0.15, # Residents
        "business_economic": 0.15, # Business & Economic Development
        "police_leadership": 0.20, # Police Leadership
        "city_government": 0.25, # City Government
        "community_advocates": 0.25 # Community Advocates & Social Services
    }

    # Create aggregate pairwise matrix weighted by stakeholder voting power
    final_pairwise_matrix = create_aggregate_pairwise_matrix(grouped_matrices, ↵
    ↪ stakeholder_weights)

    print("\n=== Final Aggregated Pairwise Matrix ===")
    print(final_pairwise_matrix)

    # Compute AHP weights from aggregated pairwise matrix
    ahp_weights, rc = ahp_method(final_pairwise_matrix, wd="mean")

    print("\n=== AHP Weights ===")
    for i in range(0, ahp_weights.shape[0]):
        print(f"{criteria[i]}: {ahp_weights[i]:.4f}")

    print(f"\nConsistency Ratio (RC): {rc:.4f}")

    weights = [ahp_weights[0], ahp_weights[1], ahp_weights[2], ahp_weights[3]]

    return weights

def test2():
    """Test 2: More complex scenario with 18 stakeholders, multiple per group.
    ↪ """
    test2_raw = {

```

```

        "participant": ["Nia Thompson", "Marcus Hill", "Priya Shah", "Caleb_
↪Johnson", "Denise Walker", "Owen Patel", "Lena Torres", "Victor Huang",_
↪Aisha Grant",
                        "Samuel Price", "Grace Kim", "Andre Coleman", "Felix_
↪Romero", "Mina Okafor", "Nathan Lee", "Riley Carter", "Fatima Noor", "Elijah_
↪Stone"],
        "stakeholder_group": ["community_residents", "community_residents",_
↪"community_residents", "community_residents", "business_economic",_
↪"business_economic",
                                "business_economic", "business_economic",_
↪"business_economic", "business_economic", "police_leadership",_
↪"police_leadership",
                                "police_leadership", "city_government",_
↪"city_government", "community_advocates", "community_advocates",_
↪"community_advocates"],
        "implementation_cost": ["High", "Medium", "Very_
↪High", "Medium", "Medium", "Low", "Medium", "High", "Medium", "Low", "Low", "Medium", "Low", "Very_
↪High", "High", "Medium",
                                "High", "Medium"],
        "expected_crime_reduction": ["Medium", "High", "Low", "Medium", "Very_
↪High", "Very High", "High", "High", "Very High", "Very High", "Very High", "Very_
↪High", "High",
                                "Medium", "High", "Low", "Medium", "Low"],
        "community_trust_impact": ["Very High", "High", "High", "Very_
↪High", "Low", "Medium", "Low", "Medium", "Medium", "Low", "Medium", "High", "Medium", "High", "High",
                                "Very High", "Very High", "Very High"],
        "implementation_time": ["Medium", "Medium", "High", "Low", "Very_
↪High", "High", "Very High", "High", "Very High", "High", "High", "High", "Very_
↪High", "Medium", "Medium",
                                "Low", "Medium", "Medium"]
    }

df = pd.DataFrame(test2_raw)
df = convert_linguistic_to_numeric(df)

# Create individual pairwise matrices grouped by stakeholder_group
grouped_matrices = create_individual_pairwise_matrix(df)

print("\n=== Grouped Pairwise Matrices by Stakeholder Group ===")
for stakeholder_group, matrix in grouped_matrices.items():
    print(f"\n{n}{stakeholder_group}:")
    print(matrix)

# Define stakeholder voting power (from scenario.json)
stakeholder_weights = {
    "community_residents": 0.15, # Residents

```

```

        "business_economic": 0.15, # Business & Economic Development
        "police_leadership": 0.20, # Police Leadership
        "city_government": 0.25, # City Government
        "community_advocates": 0.25 # Community Advocates & Social Services
    }

    # Create aggregate pairwise matrix weighted by stakeholder voting power
    final_pairwise_matrix = create_aggregate_pairwise_matrix(grouped_matrices,
↪ stakeholder_weights)

    print("\n== Final Aggregated Pairwise Matrix ==")
    print(final_pairwise_matrix)

    # Compute AHP weights from aggregated pairwise matrix
    ahp_weights, rc = ahp_method(final_pairwise_matrix, wd="mean")

    print("\n== AHP Weights ==")
    for i in range(0, ahp_weights.shape[0]):
        print(f"{criteria[i]}: {ahp_weights[i]:.4f}")

    print(f"\nConsistency Ratio (RC): {rc:.4f}")

    weights = [ahp_weights[0], ahp_weights[1], ahp_weights[2], ahp_weights[3]]

    return weights

def test3():
    """ Test 3: Even more complex scenario with 29 stakeholders, multiple per_
↪ group, and more extreme preferences."""
    test3_raw = {
        "participant": ["Jordan Ellis", "Harper Davis", "Tiana Brown", "Mateo_
↪ Garcia", "Maya Chen", "Owen Patel", "Denise Walker", "Victor Huang",
                        "Aisha Grant", "Samuel Price", "Lena Torres", "Noah_
↪ Bennett", "Isabella Wright", "Cameron Scott", "Grace Kim", "Andre Coleman",
                        "Felix Romero", "Sofia Alvarez", "Daniel Foster",_
↪ "Monica Bell", "Keith Robinson", "Mina Okafor", "Nathan Lee", "Clara Young",
                        "Ethan Brooks", "Riley Carter", "Fatima Noor", "Elijah_
↪ Stone", "Ari Morgan"],
        "stakeholder_group":_
↪ ["community_residents", "community_residents", "community_residents", "community_residents", "b
_
↪ "business_economic", "business_economic", "business_economic", "business_economic", "business_e
_
↪ "business_economic", "business_economic", "police_leadership", "police_leadership", "police_lea
_
↪ "police_leadership", "police_leadership", "police_leadership", "city_government", "city_governm

```

```

        ↪
↪ "community_advocates", "community_advocates", "community_advocates", "community_advocates"],
    "implementation_cost": ["High", "Medium", "Very
↪ High", "High", "Low", "Low", "Medium", "Medium", "Low", "Low", "Medium", "Low", "Medium", "Low", "Low",
    "Low", "Low", "Medium", "Low", "Medium", "Very
↪ High", "High", "Very High", "High", "Medium", "High", "Medium", "High"],
    "expected_crime_reduction": ["Medium", "High", "Low", "Medium", "Very
↪ High", "Very High", "Very High", "Very High", "Very High", "Very
↪ High", "High", "Very High",
    "Very High", "Very High", "Very High", "Very
↪ High", "High", "Very High", "High", "Very High", "Very
↪ High", "Medium", "High", "Medium",
    "Medium", "Low", "Medium", "Low", "Low"],
    "community_trust_impact": ["Very High", "High", "Very
↪ High", "High", "Low", "Low", "Low", "Medium", "Medium", "Low", "Low", "Low", "Low", "Medium", "Medium",
        ↪
↪ "Medium", "Medium", "Medium", "Low", "High", "High", "High", "Medium", "High", "Very
↪ High", "Very High", "Very High", "Very High"],
    "implementation_time": ["Medium", "Medium", "Low", "High", "Very
↪ High", "Very High", "Very High", "High", "Very High", "High", "Very High", "Very
↪ High", "High",
    "Very High", "High", "High", "Very
↪ High", "High", "High", "Very
↪ High", "Medium", "Medium", "Medium", "High", "Low", "Low", "Medium", "Low",
    "Medium"]
}

df = pd.DataFrame(test3_raw)
df = convert_linguistic_to_numeric(df)

# Create individual pairwise matrices grouped by stakeholder_group
grouped_matrices = create_individual_pairwise_matrix(df)

print("\n=== Grouped Pairwise Matrices by Stakeholder Group ===")
for stakeholder_group, matrix in grouped_matrices.items():
    print(f"\n{stakeholder_group}:")
    print(matrix)

# Define stakeholder voting power (from scenario.json)
stakeholder_weights = {
    "community_residents": 0.15, # Residents
    "business_economic": 0.15, # Business & Economic Development
    "police_leadership": 0.20, # Police Leadership
    "city_government": 0.25, # City Government
    "community_advocates": 0.25 # Community Advocates & Social Services
}

```

```

# Create aggregate pairwise matrix weighted by stakeholder voting power
final_pairwise_matrix = create_aggregate_pairwise_matrix(grouped_matrices,
↳ stakeholder_weights)

print("\n=== Final Aggregated Pairwise Matrix ===")
print(final_pairwise_matrix)

# Compute AHP weights from aggregated pairwise matrix
ahp_weights, rc = ahp_method(final_pairwise_matrix, wd="mean")

print("\n=== AHP Weights ===")
for i in range(0, ahp_weights.shape[0]):
    print(f"{criteria[i]}: {ahp_weights[i]:.4f}")

print(f"\nConsistency Ratio (RC): {rc:.4f}")

weights = [ahp_weights[0], ahp_weights[1], ahp_weights[2], ahp_weights[3]]

return weights

if __name__ == "__main__":
    data = load_data("./system-testing-assets/data.csv")

    if data is None:
        raise ValueError("Failed to load required data file.")

    criteria = ["cost", "crime_reduction", "community_trust",
↳ "months_to_implement"]

    criteria_types = ["min", "max", "max", "min"]

    display(data)

```

	alternative	cost	crime_reduction	community_trust	\
0	new_precinct	2500000	10.7	89	
1	expand_precinct	750000	3.8	91	
2	more_patrols	250000	2.5	71	
3	community_prevention	220000	2.6	68	

	months_to_implement
0	30
1	21
2	6
3	7

```
[14]: # Code for testing the pairwise matrix creation and AHP weight calculation
test1_weights = test1()
```

=== Grouped Pairwise Matrices by Stakeholder Group ===

business_economic:

```
[[1.          0.6          1.5          0.6          ]
 [1.66666667  1.          2.5          1.          ]
 [0.66666667  0.4          1.          0.4          ]
 [1.66666667  1.          2.5          1.          ]]
```

city_government:

```
[[1.          1.          1.          1.33333333]
 [1.          1.          1.          1.33333333]
 [1.          1.          1.          1.33333333]
 [0.75         0.75         0.75         1.          ]]
```

community_advocates:

```
[[1.          1.5          0.6          1.5          ]
 [0.66666667  1.          0.4          1.          ]
 [1.66666667  2.5          1.          2.5          ]
 [0.66666667  1.          0.4          1.          ]]
```

community_residents:

```
[[1.          1.33333333  0.8          1.33333333]
 [0.75         1.          0.6          1.          ]
 [1.25         1.66666667  1.          1.66666667]
 [0.75         1.          0.6          1.          ]]
```

police_leadership:

```
[[1.          0.4          0.66666667  0.5          ]
 [2.5          1.          1.66666667  1.25         ]
 [1.5          0.6          1.          0.75         ]
 [2.          0.8          1.33333333  1.          ]]
```

=== Final Aggregated Pairwise Matrix ===

```
[[1.          0.89104228  0.83405895  1.00118653]
 [1.1222812   1.          0.93604868  1.12361282]
 [1.19895602  1.0683205   1.          1.20037862]
 [0.99881488  0.88998628  0.83307049  1.          ]]
```

=== AHP Weights ===

cost: 0.2315

crime_reduction: 0.2598

community_trust: 0.2775

months_to_implement: 0.2312

Consistency Ratio (RC): 0.0000

```
[15]: # Test 1 TOPSIS scoring
scores = topsis_method(data[criteria], test1_weights, criteria_types, False,
    ↪False)
data['Topsis Score'] = scores

rankings = data[['alternative', 'Topsis Score']].sort_values(by='Topsis Score',
    ↪ascending=False)
print("\nTopsis Rankings:")
display(rankings)
```

Topsis Rankings:

	alternative	Topsis Score
2	more_patrols	0.575173
3	community_prevention	0.575027
1	expand_precinct	0.483431
0	new_precinct	0.423185

```
[16]: # Test 2 Pairwise matrix creation and AHP weight calculation with more
    ↪stakeholders and multiple stakeholders per group
test2_weights = test2()
```

=== Grouped Pairwise Matrices by Stakeholder Group ===

business_economic:

```
[[1.          0.59235304 1.12246205 0.61479778]
 [1.68818243 1.          1.89492071 1.03789082]
 [0.89089872 0.52772657 1.          0.54772256]
 [1.62655108 0.96349248 1.82574186 1.          ]]
```

city_government:

```
[[1.          1.29099445 1.11803399 1.49071198]
 [0.77459667 1.          0.8660254 1.15470054]
 [0.89442719 1.15470054 1.          1.33333333]
 [0.67082039 0.8660254 0.75 1.          ]]
```

community_advocates:

```
[[1.          1.44224957 0.66038545 1.25992105]
 [0.69336127 1.          0.4578857 0.87358046]
 [1.51426716 2.18395116 1.          1.90785707]
 [0.79370053 1.14471424 0.52414828 1.          ]]
```

community_residents:

```
[[1.          1.25743343 0.81903626 1.25743343]
 [0.79527073 1.          0.65135556 1.          ]]
```



```
[1.22094717 1.53525978 1.          1.53525978]
[0.79527073 1.          0.65135556 1.          ]]
```

```
police_leadership:
[[1.          0.49324241 0.69336127 0.53132928]
 [2.02740067 1.          1.40572111 1.07721735]
 [1.44224957 0.71137866 1.          0.76630943]
 [1.88207206 0.92831777 1.30495588 1.          ]]
```

```
=== Final Aggregated Pairwise Matrix ===
[[1.          0.97031585 0.85069966 0.99252079]
 [1.03059226 1.          0.87672449 1.02288424]
 [1.17550299 1.14060918 1.          1.16671116]
 [1.00753557 0.97762773 0.85711017 1.          ]]
```

```
=== AHP Weights ===
cost: 0.2373
crime_reduction: 0.2446
community_trust: 0.2790
months_to_implement: 0.2391
```

Consistency Ratio (RC): -0.0000

```
[17]: # Test 2 TOPSIS scoring
scores = topsis_method(data[criteria], test2_weights, criteria_types, False,
    ↪False)
data['Topsis Score'] = scores

rankings = data[['alternative', 'Topsis Score']].sort_values(by='Topsis Score',
    ↪ascending=False)
print("\nTopsis Rankings:")
display(rankings)
```

Topsis Rankings:

	alternative	Topsis Score
2	more_patrols	0.595991
3	community_prevention	0.595570
1	expand_precinct	0.496335
0	new_precinct	0.402457

```
[18]: # Test 3 Pairwise matrix creation and AHP weight calculation with more
    ↪stakeholders and multiple stakeholders per group
test3_weights = test3()
```

```
=== Grouped Pairwise Matrices by Stakeholder Group ===
```

```
business_economic:
[[1.          0.48104698 1.04137974 0.50300175]
 [2.07879902 1.          2.16481919 1.04563955]
 [0.9602645  0.46193234 1.          0.48301473]
 [1.98806464 0.9563525  2.07033025 1.          ]]
```

```
city_government:
[[1.          1.38726382 1.20140571 1.53525978]
 [0.72084342 1.          0.8660254  1.10668192]
 [0.83235829 1.15470054 1.          1.27788621]
 [0.65135556 0.903602  0.78254229 1.          ]]
```

```
community_advocates:
[[1.          1.56508458 0.69282032 1.41421356]
 [0.6389431  1.          0.44267277 0.903602  ]
 [1.44337567 2.25900501 1.          2.04124145]
 [0.70710678 1.10668192 0.48989795 1.          ]]
```

```
community_residents:
[[1.          1.35120015 0.88011174 1.35120015]
 [0.7400828  1.          0.65135556 1.          ]
 [1.13621937 1.53525978 1.          1.53525978]
 [0.7400828  1.          0.65135556 1.          ]]
```

```
police_leadership:
[[1.          0.50724322 0.77416857 0.58156398]
 [1.97144085 1.          1.52622754 1.14651899]
 [1.29170834 0.6552103  1.          0.75121105]
 [1.71950126 0.87220535 1.33118383 1.          ]]
```

```
=== Final Aggregated Pairwise Matrix ===
[[1.          0.99346723 0.89570856 1.02786741]
 [1.00657573 1.          0.9015985  1.03462639]
 [1.11643457 1.10914115 1.          1.14754671]
 [0.97288813 0.96653247 0.87142423 1.          ]]
```

```
=== AHP Weights ===
cost: 0.2441
crime_reduction: 0.2458
community_trust: 0.2726
months_to_implement: 0.2375
```

Consistency Ratio (RC): 0.0000

```
[19]: # Test 3 TOPSIS scoring
scores = topsis_method(data[criteria], test3_weights, criteria_types, False,
↪False)
```

```

data['Topsis Score'] = scores

rankings = data[['alternative', 'Topsis Score']].sort_values(by='Topsis Score',
    ↪ascending=False)
print("\nTopsis Rankings:")
display(rankings)

```

Topsis Rankings:

	alternative	Topsis Score
2	more_patrols	0.598967
3	community_prevention	0.598855
1	expand_precinct	0.500915
0	new_precinct	0.399405